CIS*3110 Operating Systems

Week 3

Lecture #5 Virtualization: More scheduling

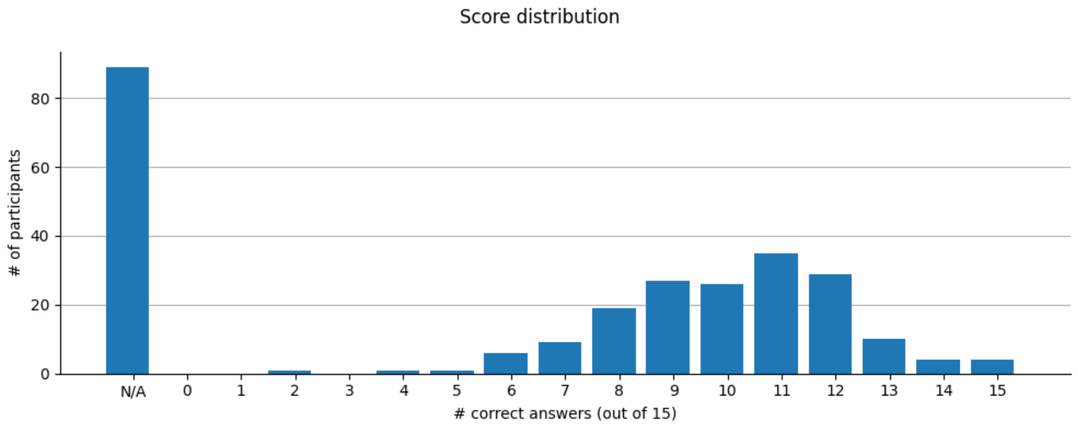
Prof. Yulia Kotseruba

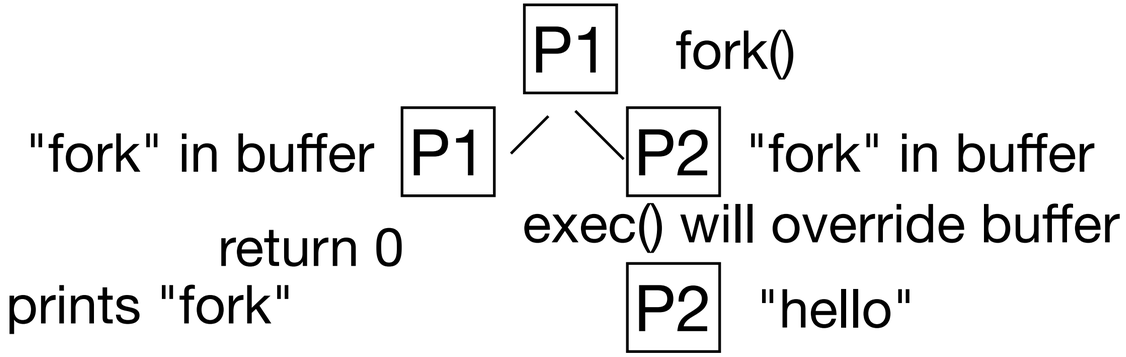
January 22, 2026

Quiz 3

```
int rc = fork();
printf("fork ");  fflush(stdout);
if (rc == 0) {
        char* args[] = {"echo","hello",NULL};
        execvp(args[0],args);
        printf("child \n");
};
return 0;
```

- **67%** average (down from 71% in quiz 2)
- 3 hardest questions:
  - (22%) What is the output of this code? a) fork child b) fork fork child c) fork fork hello child d) fork fork hello
  - (48%) Which command will list all files all files and directories in CWD, with their sizes and permissions? a) top b) ls -la c) ls -l d) ps aux
  - (48%) Which command will make "myscript.sh" executable by only you? a) chmod a+x myscript.sh b) chown a+x myscript.sh c) touch a+x myscript.sh d) chmod u+x myscript.sh





without fflush() will print "fork hello"

with fflush() will print "fork fork hello"

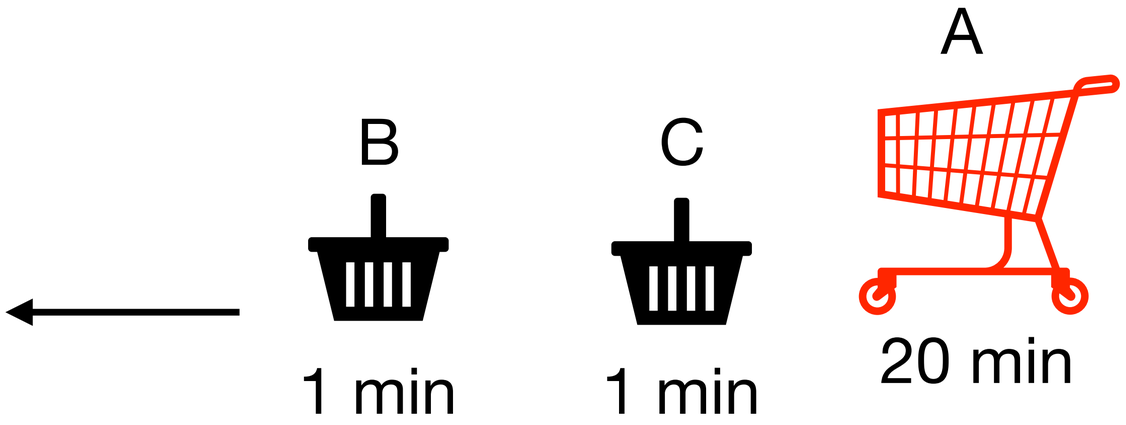
Recap: Scheduling

- Metrics:
  - **Response time** - how quickly is the process scheduled after arrival
  - **Turnaround** - how quickly the process finishes after arrival
- Scheduling policies
  - Non-preemptive:
    - **FIFO** — simple but ineffective
    - **SJF** — provably optimal for turnaround under not very realistic conditions, poor response time
  - Preemptive
    - **PSJF** — provably optimal for turnaround under not very realistic conditions, poor response time
    - **RR** — better response time at the expense of turnaround

Recap

- Processes
  - ~~Are equal in duration~~
  - Arrive at the same time
  - Run to completion
  - Run on CPU only (no I/O)
  - Have a known run-time

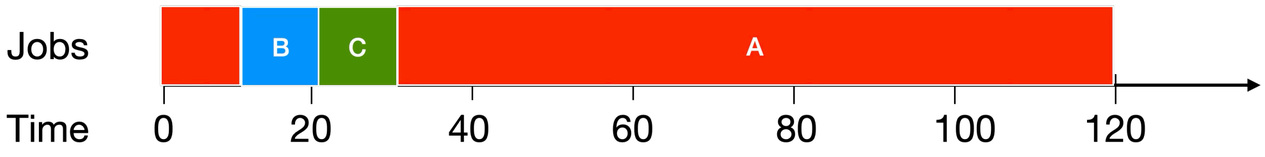
Which algorithm is optimal under these conditions? SJF for turnaround



Recap

- Processes
  - ~~Are equal in duration~~
  - ~~Arrive at the same time~~
  - ~~Run to completion~~
  - Run on CPU only (no I/O)
  - Have a known run-time

Which algorithm is optimal under these conditions? PSJF for turnaround

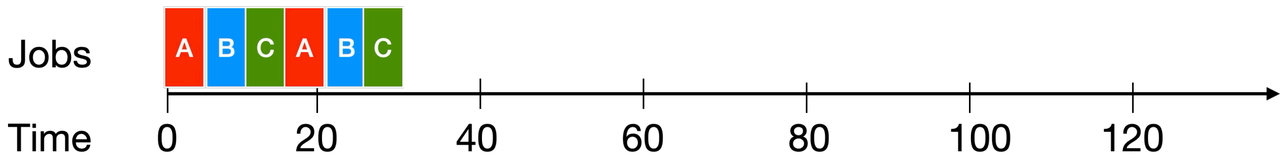


Recap

- Processes
  - ~~Are equal in duration~~
  - ~~Arrive at the same time~~
  - ~~Run to completion~~
  - Run on CPU only (no I/O)
  - Have a known run-time

Which algorithm has the best response time under these conditions?

Round robin (with a small quantum)



What is RR with a very large quantum?

FIFO

No more assumptions

- Processes
  - ~~Are equal in duration~~
  - ~~Arrive at the same time~~
  - ~~Run to completion~~
  - ~~Run on CPU only (no I/O)~~
  - ~~Have a known run-time~~

Now we don’t assume anything about the processes

How can we minimize turnaround and response?

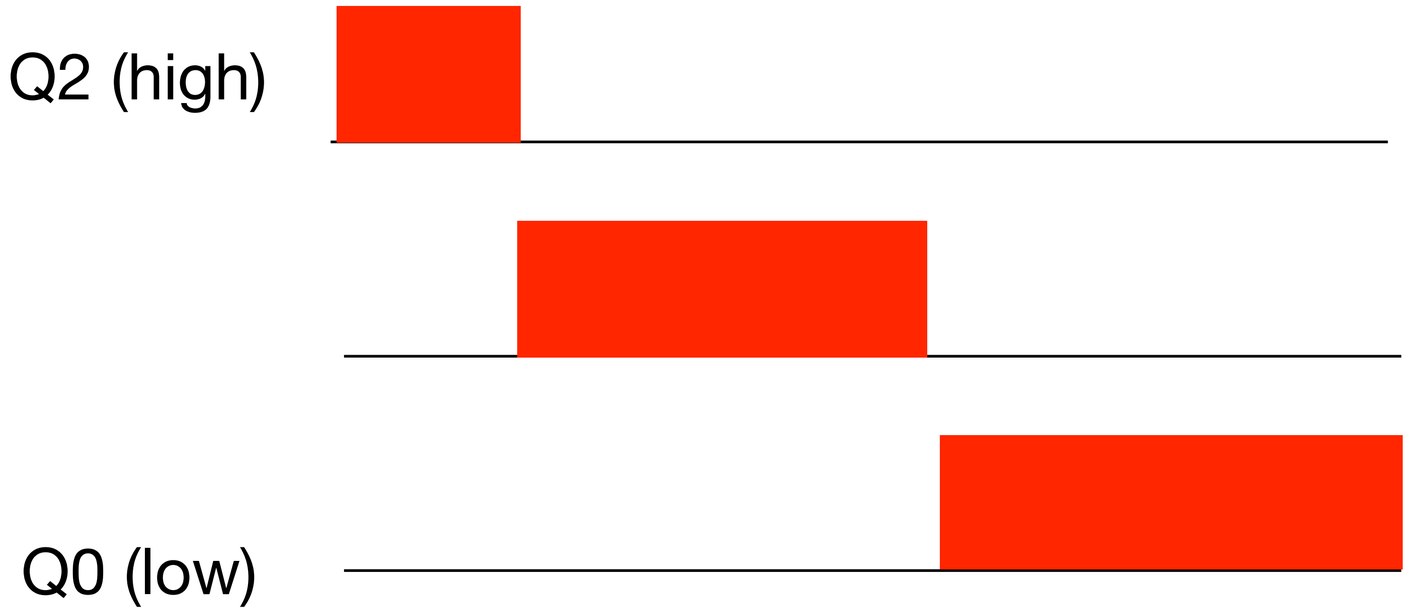
MLFQ

- We don’t know anything about processes but want to minimize turnaround and response time?
- Solution: **MLFQ** - Multi-Level Feedback Queue
  - Instead of one queue, has **several queues** with different **priorities**
  - Processes start in the highest priority queue
  - If they run too long, they are moved to a lower priority queue
  - Basic operation:
    - Run all processes in the priority queue
    - Then run all processes in the lower priority queues

MLFQ

how long should the process run in a queue before being demoted to a lower priority queue?

each queue gets an **allotment** - fixed time that the process can run in this queue



new process arrives and is placed in the highest priority queue (assume it will finish soon)

process is still running

process priority reduced

process is still running

process priority reduced

process finishes

MLFQ

- New jobs start at the highest priority
- Higher-priority processes preempt lower-priority ones
- Each queue has an **allotment** - max time any process can spend in the queue
  - can be the same in all queues
  - if different, shorter allotments in higher priority queues
- MLFQ assumes that new jobs are going to be short and gives them higher priority (like SJF)
- If the job takes longer and uses up its allotment, it is moved to a lower priority queue

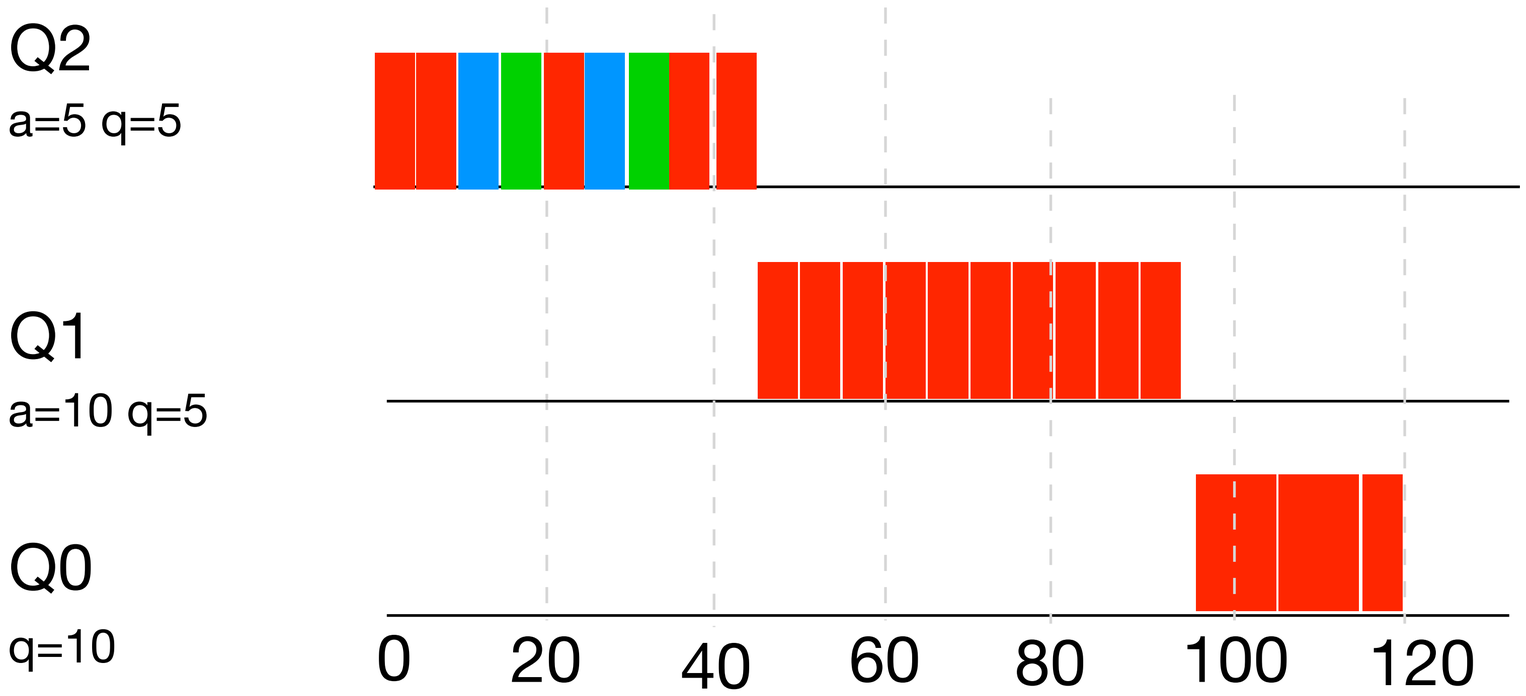
MLFQ

- More rules
  - Each queue can have a **different scheduler**
  - Common choices are **RR** and **FIFO**
  - RR can also have **different quantums** at different priority levels
    - usually higher priority queues have shorter quantums to increase response time
  - FIFO is often used for the lowest priority queue

MLFQ

Duration: A=100s, B=C=10s

Arrival: A=0s, B=C=10s



Max duration the process can stay in each queue is q*a

What are the turnaround times?

A $T_{turnaround} = 120 - 0 = 120$

B $T_{turnaround} = 30 - 10 = 20$

C $T_{turnaround} = 35 - 10 = 25$

$$Avg_T_{turnaround} = \frac{120 + 20 + 25}{3} = 55$$

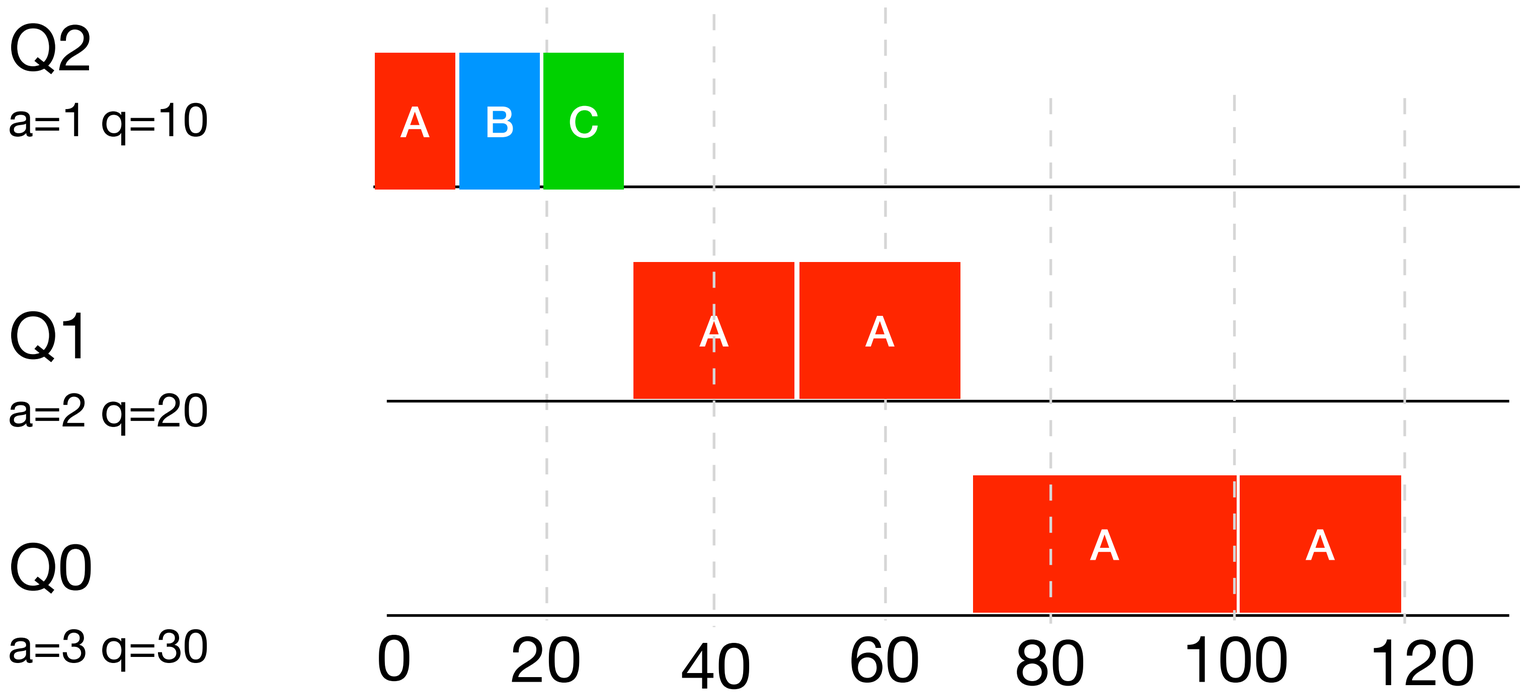
What is avg response time?

$$Avg_T_{response} = \frac{0 + 0 + 5}{3} = 1.66$$

MLFQ

Duration: A=100s, B=C=10s

Arrival: A=0s, B=C=10s



Max duration the process can stay in each queue is q*a

What are the turnaround times?

A $T_{turnaround} = 120 - 0 = 120$

B $T_{turnaround} = 20 - 10 = 10$

C $T_{turnaround} = 30 - 10 = 20$

$$Avg_T_{turnaround} = \frac{120 + 10 + 20}{3} = 50$$

What is avg response time?

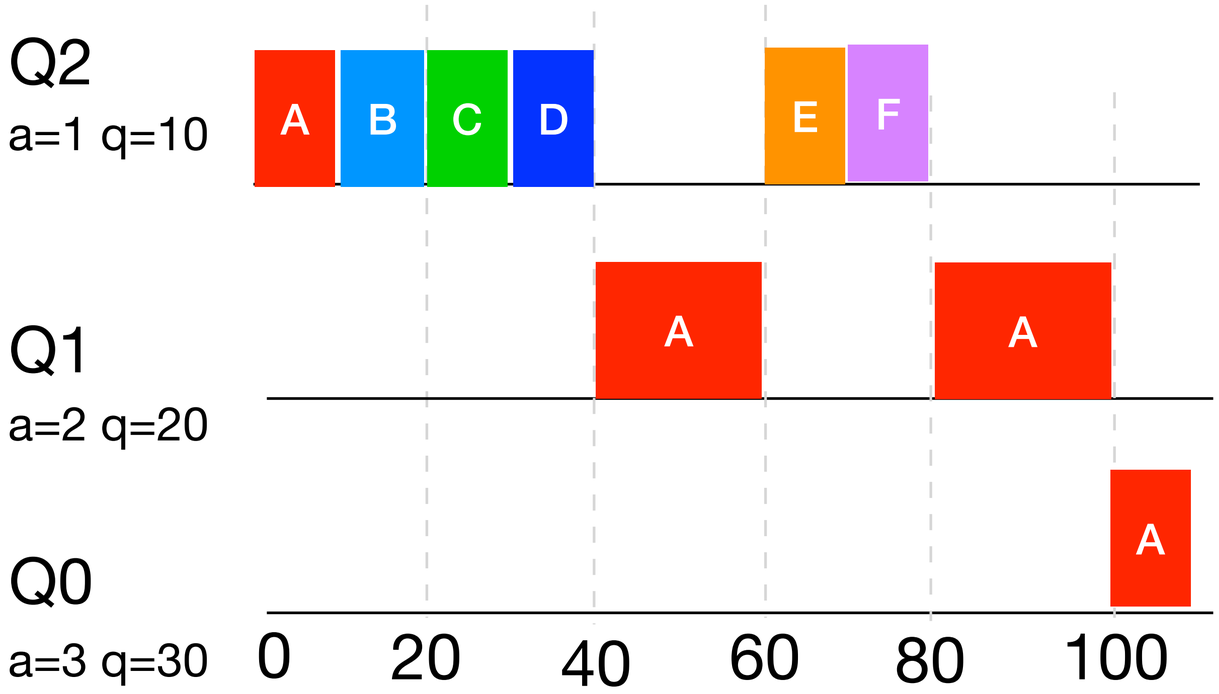
$$Avg_T_{response} = \frac{0 + 0 + 10}{3} = 3.33$$

changing these settings improved turnaround by 5 but doubled response time

MLFQ

Duration: A=60s, B=C=D=E=F=10s

Arrival: A=0s, B=5s, C=10s, D=15s, E=50s, F=60s



Starvation — too many short processes consume all CPU and long-running processes never receive any CPU share (starve)

Solution: bump up all processes to highest priority at fixed time intervals

A ***starves*** because short processes use up all CPU

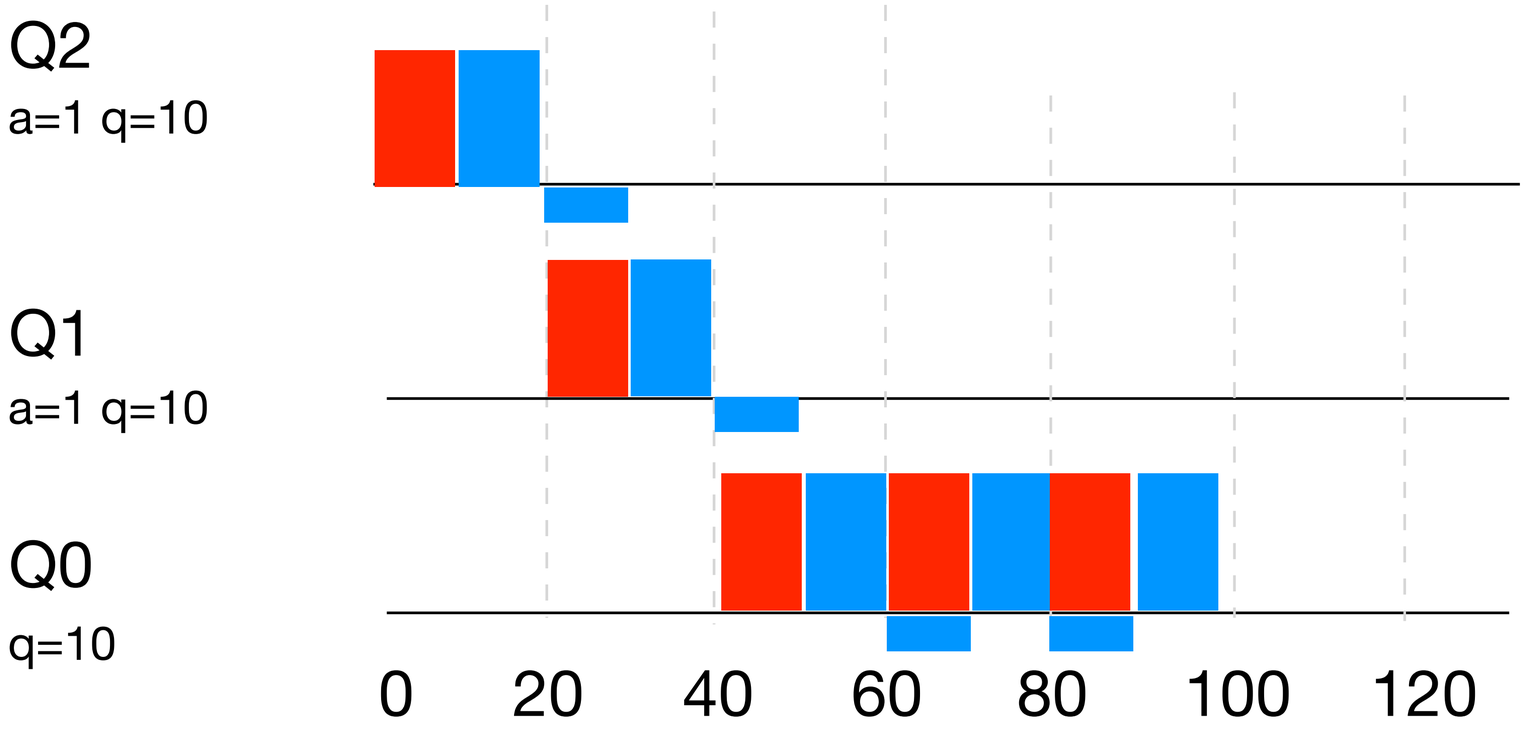
MLFQ + I/O

- How to treat I/O?
  - Option #1 - if a process sends an I/O request before its allotment is up, keep its priority level — can be gamed by sending an I/O request just before the allotment runs out
  - Option #2 - count both CPU and I/O towards the allotment and reduce priority

MLFQ + I/O

Duration: A=B=50 B issues 10s I/O every 10s

Arrival: A=B=0s



MLFQ

- 5 rules
  - If priority(A) > priority(B), A runs and B doesn't
  - If priority(A) = priority B, they run in the order determined by the scheduler of that queue (usually RR)
  - When the job first arrives, it's placed at the highest priority
  - Once job uses up the allotment of its queue, it's demoted to the lower priority queue
  - Bump up priority of all processes at some time interval to avoid starvation

MLFQ

- Parameters
  - **number of queues**
  - queue **allotments**
  - **scheduling algorithm** for each queues + scheduler's parameters (e.g. quantum for round robin)
  - **when to demote** a process to a lower-priority queue
    - whether to count I/O time towards allotment
  - **when to upgrade** a process to a higher priority queue

MLFQ summary

- The **most general CPU scheduling algorithm** - can fit many workloads
- Multiple queues with different priorities and allotments
- **Feedback is used to adjust priority**
  - If a process runs too long or issues too many I/O requests, it is demoted
- Can be tuned to avoid **starvation**
- Has **too many parameters**
- **Tuning** parameters for optimal performance is **not easy**

MLFQ in the wild

- MLFQ is a base scheduler in these operating systems:
  - BSD Unix
  - Solaris
  - Windows NT

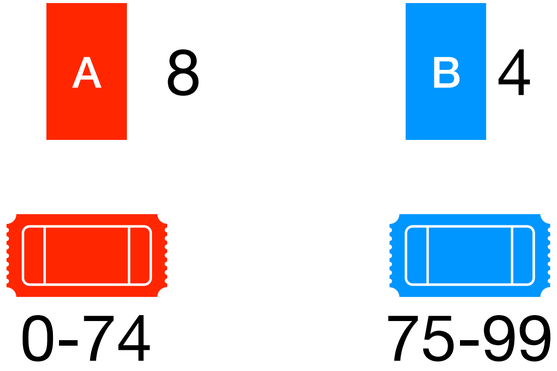
Fair-share schedulers

- What if we don't know anything about processes but want to optimize fairness?
- **Fairness** - each process gets a **proportional share of CPU** resources
- Solutions:
  - **lottery scheduling**
  - **stride scheduling**
  - **Linux Completely Fair Scheduler** (CFS)
- Note: round robin is fair-share for processes on the ready queue

Lottery scheduling

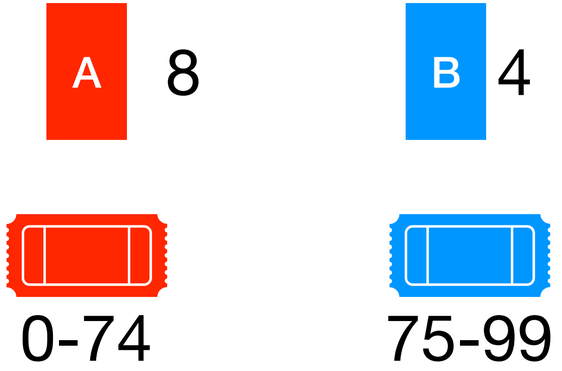
- Each process gets lottery tickets that represent CPU share
- The scheduler runs a **lottery at fixed time intervals**
  - Picks one winning ticket at **random**
  - The process holding the winning ticket runs until the next lottery

Lottery scheduling



Random number:

Lottery scheduling

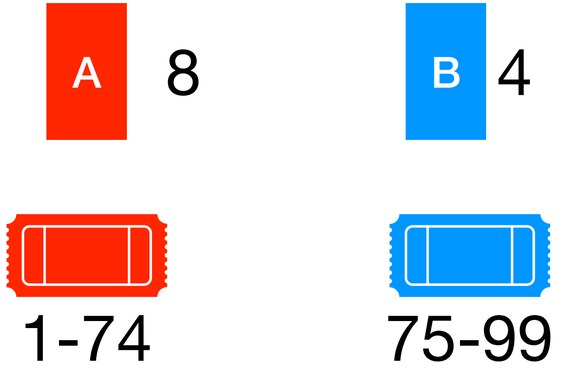


Random number: 511275

511275%100 = 75

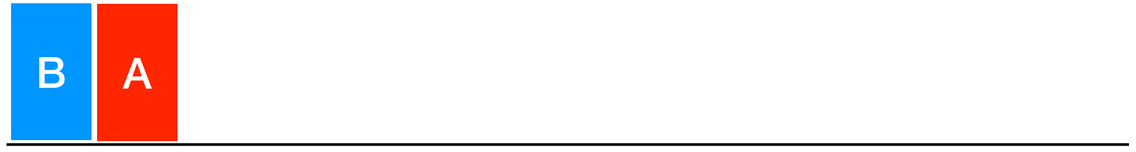
pick a large random number and do a modulo with the total number of tickets to determine the winner

Lottery scheduling

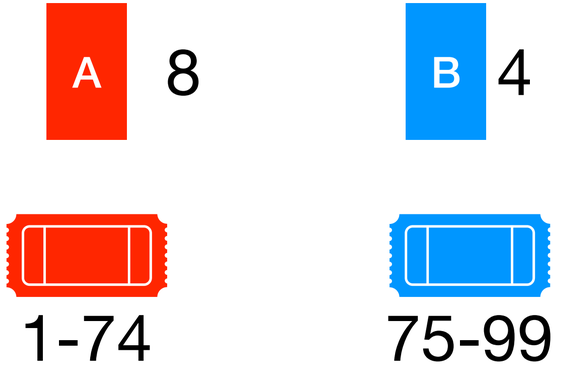


Random number: 404934

404934%100 = 34



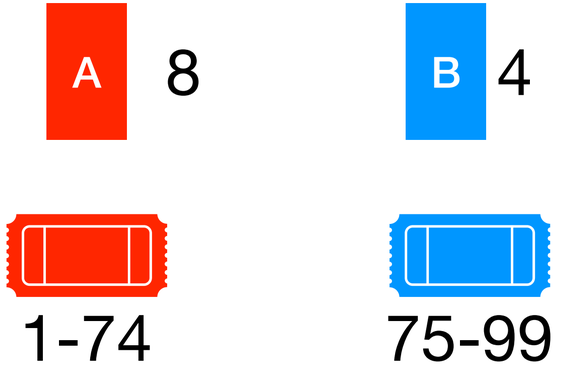
Lottery scheduling



Random number: 783799

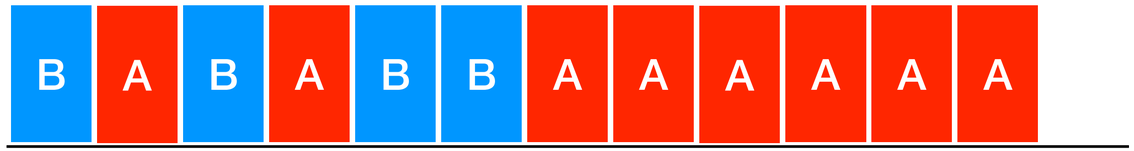
783799%100 = 99

Lottery scheduling



Random number: 250506

250506%100 = 6



we cannot guarantee that every interval will be fairly divided, e.g. you can get a streak of A's

but in the long run it will be fair

Lottery scheduling

- The good
  - **Very simple** to implement
    - All you need is a list to store jobs and tickets and a random number generator
  - **Randomness is fair**
- The bad
  - Randomness does **not guarantee** exact proportional share
  - Proper **random number generator is hard** to implement
  - How do we **assign tickets**? Let the users decide?

Stride scheduling

- Stride scheduler is a **deterministic fair-share** scheduler
- Very similar to lottery scheduler but sans randomness
- Each job has three properties
  - **tickets** - determine share of CPU
  - **stride** - inversely proportional to the number of tickets
  - **pass** - keeps track of the job's progress
- Always pick the process that has the **lowest pass value** to run next
- Whenever the process runs, its **pass is incremented** by the stride

Stride scheduling

A	s=100 p=0	B	s=200 p=0	C	s=40 p=0

always pick the one with the lowest pass to run next

---

each selected process runs for 1 quantum until its CPU burst time runs out

Stride scheduling

A	s=100 p=100	B	s=200 p=0	C	s=40 p=0

always pick the one with the lowest pass to run next

each selected process runs for 1 quantum until its CPU burst time runs out

Stride scheduling

A	s=100 p=100	B	s=200 p=200	C	s=40 p=0

always pick the one with the lowest pass to run next

each selected process runs for 1 quantum until its CPU burst time runs out

Stride scheduling

A	s=100 p=100	B	s=200 p=200	C	s=40 p=40

always pick the one with the lowest pass to run next

each selected process runs for 1 quantum until its CPU burst time runs out

Stride scheduling

A	s=100 p=100	B	s=200 p=200	C	s=40 p=80

always pick the one with the lowest pass to run next

each selected process runs for 1 quantum until its CPU burst time runs out

Stride scheduling

A	s=100 p=100	B	s=200 p=200	C	s=40 p=120

always pick the one with the lowest pass to run next

each selected process runs for 1 quantum until its CPU burst time runs out

Stride scheduling

A	s=100 p=200	B	s=200 p=200	C	s=40 p=120

always pick the one with the lowest pass to run next

each selected process runs for 1 quantum until its CPU burst time runs out

Stride scheduling

A	s=100 p=200	B	s=200 p=200	C	s=40 p=160

always pick the one with the lowest pass to run next

each selected process runs for 1 quantum until its CPU burst time runs out

Stride scheduling

- Problems:
  - difficult to assign **strides for new processes**
    - if set too high, they will never run
    - if set too low, will monopolize the CPU
  - lottery scheduling avoids this by not having a **global state** per process (stride)
- **Ticket assignment** is still a problem in both
- Both have **low system overhead**

Linux Completely Fair Scheduler (CFS)

- Linux Completely Fair Scheduler
  - fairly divides CPU among all competing processes
  - does not use a fixed time slice for running processes
  - prioritizes efficiency for making scheduling decisions

CFS: Basic operation

- Basic operation:
  - Until there are jobs left in the runqueue
    - pick a job with that used the least CPU time
    - run it for some time
    - increment the job’s time
- we want jobs that ran for a while to give way to jobs that did not get much CPU time

CFS: Parameters

- **nice** - process priority
- **sched_latency** - time interval within which processes currently in the runqueue will get a share of the CPU proportional to the priority (nice)
- **vruntime** - virtual runtime - a representation of how long the process used up the CPU, weighted by the process's priority (nice)

CFS: nice

- Each process has a **priority** value called **nice**
  - an **integer** between -20 (highest priority) and 19 (lowest priority)
  - why nice? jobs with lower priority are *nicer* to others, so their nice value is higher
- nice value determines two things:
  - how long the process will run - high priority get to run longer
  - how much its vruntime will increase
    - high-priority job's vruntime increases slower so it are more likely to get selected again

CFS: CPU share

- CPU time is divided into fixed time slices — **sched_latency**
- each sched_latency slice is divided fairly among the processes in the run queue
  - share of the CPU time depends on priority (nice value)
    - higher priority processes get a larger share
    - lower priority processes get a smaller share
  - implemented as a lookup table prio_to_weight

CFS: CPU share

- Assume n processes and fixed sched_latency
- Time slice of each process is proportional to their weight
- Weight is calculated based on the process's nice value using a lookup table

```
static const int prio_to_weight[40] = {
 /* -20 */     88761,     71755,     56483,     46273,     36291,
 /* -15 */     29154,     23254,     18705,     14949,     11916,
 /* -10 */      9548,      7620,      6100,      4904,      3906,
 /*  -5 */      3121,      2501,      1991,      1586,      1277,
 /*   0 */      1024,       820,       655,       526,       423,
 /*   5 */       335,       272,       215,       172,       137,
 /*  10 */       110,        87,        70,        56,        45,
 /*  15 */        36,        29,        23,        18,        15,
};
```

https://github.com/torvalds/linux/blob/v2.6.34/kernel/sched.c#L1356

table definition in the Linux kernel

$$\text{time_slice}_k = \frac{\text{weight}_k}{\Sigma^i \text{weight}_i} \cdot \text{sched_latency}$$

time slice for each process k is its weight divided by the weight of all processes multiplied by sched_latency

Example: 3 jobs with different priorities

Nice	-20	-5	5
weight	88761	3121	335
time_slice	0.96	0.03	0.01

a job with the highest priority will use 96% of CPU time within sched_latency

CFS: CPU share

- What if there are too many jobs or too many low priority jobs?
  - min_granularity sets a minimum time slice for each process
  - sched_latency will be increased to accommodate all of them

CFS: vruntime

- **vruntime** (virtual runtime) - a weighted time a task has run on CPU
- Each new process gets assigned the **smallest vruntime among all tasks**
- vruntime of a running process is increased by the amount **proportional to their priority**
  - lower priority get a larger increase -> less likely to be selected again
  - higher priority get a smaller increase -> more likely to be selected again

CFS: vruntime

- **vruntime** - a weighted time a task has run on CPU (in nanoseconds)
- for each process vruntime is incremented using this formula:

current vruntime

amount of actual CPU time the process ran

$$\text{vruntime}_i = \text{vruntime}_i + \frac{1024}{\text{weight}_i} \cdot \text{runtime}_i$$

weight depending on priority (nice value)

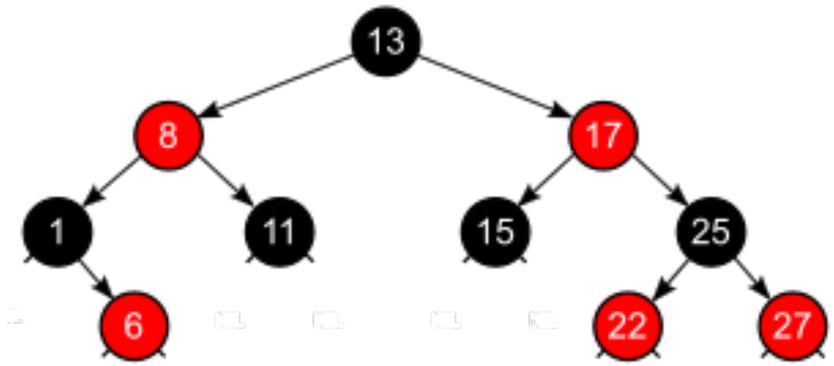
Nice	-20	-5	5
weight	88761	3121	335
runtime	20	20	20
vruntime increment	0.23	6.56	60.13

jobs with higher priority would get a very small vruntime increment and more likely to be chosen to run again

CFS: Basic operation

- Basic operation:
  - Until there are jobs left in the runqueue
    - pick a job with that used the least CPU time, i.e. has the **smallest vruntime**
    - run it for some time within **sched_latency**
      - time is proportional to the process's niceness (priority), but cannot be smaller than min_granularity
    - increment the job's CPU time (vruntime)
      - vruntime increase is **inversely proportional to the process niceness**, i.e. higher priority jobs will see a smaller increase and will be more likely to run again

CFS: efficiency



- CFS maintains a **red-black tree sorted by vruntime**
  - red-black tree is a self-balancing binary search tree
  - when any of the nodes are modified, the tree is rebalanced
  - search, insert and delete are O(log n)
  - not perfectly balanced, but is the best choice for cases when there are a lot of insertions/deletions (as would be in scheduling)
- leftmost node in the tree is always the one with the smallest vruntime
  - picking the next process to run is always guaranteed to be O(1)

CFS: Basic operation

- Basic operation:
  - Until there are jobs left in the runqueue
    - pick a job with that used the least CPU time, i.e. has **the smallest vruntime**
      - pick the **leftmost node** in RBTree in O(1)
    - run it for some time
      - time is proportional to the process's niceness (priority) within a sched_latency, but cannot be smaller than min_granularity
    - increment the job's CPU time (vruntime)
      - vruntime increase is **inversely proportional to the process niceness**, i.e. higher priority jobs will see a smaller increase and will be more likely to run again
      - process is **re-inserted into the RBTree in O(log n)** based on the **new vruntime,** which pushes other tasks to the left

CFS and stride scheduling

- **vrutime** is similar to **pass** in stride scheduling but more flexible
- both aim for low overhead and fairness

EEVDF

- CFS **was** the default Linux scheduler from kernel v2.6 (2007) to v6.6 (2024)
- Since kernel version 6.6, **Earliest Eligible Virtual Deadline First (EEVDF)*** replaces CFS
  - CFS does not handle well **latency requirements**
    - if a process needs CPU fast, it cannot express this to the scheduler
    - priority (nice) may give it more CPU time but there is little control **when**
    - it is possible to reduce latency via other means but at the risk of messing up other processes

*first described in 1995 paper by Stoica and Abdel-Wahab

EEVDF

- Similar to CFS in operation, also uses **RBTree**
- Each process gets a share of CPU proportional to their nice value
  - If a process didn’t get it’s share, it gets a **"lag"** - amount it was shortchanged
  - Only processes with **positive lag** (owed CPU time) are **eligible** to be scheduled
  - Tasks that got their share will not be scheduled until others catch up
- Out of eligible processes, those with earliest **virtual deadline** are selected
  - Virtual deadline = eligible time + CPU share proportional to priority
  - Eligible time - time when it will catch up with others

EEVDF

- EEVDF is better for latency-critical tasks (aka interactive processes)
- Interactive tasks have a small nice value and shorter CPU share
- Their virtual deadlines are earlier than CPU-intensive tasks, even if the priorities are the same

- If you have Linux, check the kernel number using `uname -r`
  - if < 6.6 you have CFS, otherwise EEVDF

Summary

- We are done with CPU scheduling
- Four new schedulers: lottery, stride, CFS, EEVDF
- All aim for **fairness**
  - lottery and stride - global fairness
  - CFS and EEVDF - fairness within smaller time windows
- All aim for **efficiency**
  - lottery and stride - few parameters and list for managing the queue
  - CFS and EEVDF - more parameters, use RBTree to manage the queue

Reading

- Chapter 8 Scheduling: The MLFQ
  - https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-mlfq.pdf
- Chapter 7 Scheduling: Proportional share
  - https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched-lottery.pdf
- Code
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/cpu-sched-mlfq
  - https://github.com/remzi-arpacidusseau/ostep-homework/tree/master/cpu-sched-lottery
- Optional reading
  - https://www.kernel.org/doc/Documentation/scheduler/sched-design-CFS.txt
  - https://lwn.net/Articles/925371/
  - https://github.com/jamesyoung-15/Kernel-Benchmark-CFS-EEVDF/blob/main/README.md

Quiz 4

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet